A4M

KONCHADA. SHYAM SAI(RA2311026010305)

Recap of Review 1

- **PROBLEM STATEMENT** – TO ESTIMATE THE ENERGY NEEDED TO CREATE THE TRANSPARENT CRYSTAL CONDUCTOR AND INFERE ITS STABLILIY OF EXISTENCE
- OBJECTIVE - Model to predict the Formation energy and classify whether the material or cluster of molecules are stable or unstable
- PROPOSED IDEA : XGBoost – Regressor & XGBoost Classification
- The dataset used is Nomad 2018 predicting Transparent conductor
- Dataset link : https://www.kaggle.com/competitions/nomad2018-predict-transparent-conductors/data?select=train.zip
- Dataset size : 2400 (datapoints) * 14 (features)

Mathematical Modeling

XGBoost Regressor

Semantic math colors — consistent roles: predictions, trees, grads, regularizers, leaf weights.

Color legend: $\hat{y}$ Predictions/output f_k Tree functions $\mathcal{L}$ Objective/loss g, h Gradient/Hessian γ, λ Regularizers w_j Leaf weights

1 Model (Core Idea)

- **Additive ensemble:**

$$\hat{y}_i = \sum_{k=1}^{K} f_k(\mathbf{x}_i), \qquad f_k \in \mathcal{F}$$

- Each tree is piecewise-constant: $f(\mathbf{x}) = w_{q(\mathbf{x})}$ where $q(\cdot)$ maps samples to leaf indices.

2 Objective (Regularized)

$$\mathcal{L} = \sum_{i=1}^{n} \ell(y_i, \hat{y}_i) + \sum_{k=1}^{K} \Omega(f_k)$$

Typical tree regularizer:

$$\Omega(f) = \gamma T + \tfrac{1}{2}\lambda \sum_{j=1}^{T} {w_j}^2$$

where $T = \#$ leaves, γ penalizes extra leaves, λ performs L2 on leaf weights.

3 Additive / Stage-wise Training

At iteration t we add f_t. Use second-order Taylor expansion at current predictions $\hat{y}_i^{(t-1)}$.

Derivatives:

$$g_i = \left.\frac{\partial \ell(y_i, \hat{y})}{\partial \hat{y}}\right|_{\hat{y}=\hat{y}_i^{(t-1)}}, \quad h_i = \left.\frac{\partial^2 \ell(y_i, \hat{y})}{\partial \hat{y}^2}\right|_{\hat{y}=\hat{y}_i^{(t-1)}}$$

Approximate incremental objective (drop constants):

$$\tilde{\mathcal{L}}^{(t)} = \sum_{i=1}^{n} \left[g_i f_t(\mathbf{x}_i) + \tfrac{1}{2} h_i f_t(\mathbf{x}_i)^2 \right] + \Omega(f_t).$$

4 Leaf weight (fixed structure)

For leaf j with samples I_j:

$$G_j = \sum_{i \in I_j} g_i, \quad H_j = \sum_{i \in I_j} h_i$$

Optimal leaf weight:

$$w_j^\star = -\frac{G_j}{H_j + \lambda}.$$

Leaf score (objective contribution):

$$\text{Score}(I_j) = -\tfrac{1}{2} \frac{{G_j}^2}{H_j + \lambda}.$$

5 Split gain (candidate split)

Parent $I \to I_L, I_R$:

$$\text{Gain} = \tfrac{1}{2}\left(\frac{{G_L}^2}{H_L + \lambda} + \frac{{G_R}^2}{H_R + \lambda} - \frac{{G_I}^2}{H_I + \lambda} \right) - \gamma.$$

Accept splits with positive (sufficient) gain subject to `min_child_weight`, `max_depth`, etc.

Mathematical Modeling

XGBoost Classifier

Color legend: $\hat{p}$ Predicted probability $z = \sum f_k$ Logit / score f_k Tree outputs $\mathcal{L}$ Objective / log-loss g, h Gradient / Hessian γ, λ Regularizers w_j Leaf weights

1 Model (Core Idea)

- **Additive ensemble (logit space):**

$$z_i = \sum_{k=1}^{K} f_k(\mathbf{x}_i), \qquad f_k \in \mathcal{F}$$

- Probability via sigmoid:

$$\hat{p}_i = \sigma(z_i) = \frac{1}{1 + e^{-z_i}}$$

- Decision: $\hat{y}_i = \mathbf{I}(\hat{p}_i \geq \tau)$ for threshold τ.

2 Objective (Regularized)

Binary cross-entropy (logistic) + regularizer:

$$\mathcal{L} = -\sum_{i=1}^{n} \left[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \right] + \sum_{k=1}^{K} \Omega(f_k).$$

Regularizer (tree complexity):

$$\Omega(f) = \gamma T + \tfrac{1}{2} \lambda \sum_{j=1}^{T} {w_j}^2.$$

3 Stage-wise training (Taylor approx)

At iteration t, current logit is $z_i^{(t-1)}$ and $\hat{p}_i^{(t-1)} = \sigma(z_i^{(t-1)})$. Derivatives of loss w.r.t. logit:

$$g_i = \left. \frac{\partial \ell}{\partial z_i} \right|_{z^{(t-1)}}, \qquad h_i = \left. \frac{\partial^2 \ell}{\partial {z_i}^2} \right|_{z^{(t-1)}}.$$

Second-order approx for f_t (drop constants):

$$\widetilde{\mathcal{L}}^{(t)} = \sum_{i=1}^{n} \left[g_i f_t(\mathbf{x}_i) + \tfrac{1}{2} h_i f_t(\mathbf{x}_i)^2 \right] + \Omega(f_t).$$

4 Gradients & Hessians (logistic loss)

For logistic loss:

$$\hat{p} = \sigma(z), \qquad g = \hat{p} - y, \qquad h = \hat{p}(1 - \hat{p}).$$

Concretely at iteration $t - 1$:

$$g_i = \hat{p}_i^{(t-1)} - y_i, \qquad h_i = \hat{p}_i^{(t-1)}(1 - \hat{p}_i^{(t-1)}).$$

5 Leaf weight (fixed structure)

For leaf j with samples I_j:

$$G_j = \sum_{i \in I_j} g_i, \qquad H_j = \sum_{i \in I_j} h_i.$$

Optimal leaf weight (logit-space):

$$w_j^* = -\frac{G_j}{H_j + \lambda}.$$

Leaf score (objective reduction):

$$\text{Score}(I_j) = -\tfrac{1}{2} \frac{{G_j}^2}{H_j + \lambda}.$$

6 Split gain (candidate split)

Parent $I \to I_L, I_R$:

$$\text{Gain} = \tfrac{1}{2} \left(\frac{{G_L}^2}{H_L + \lambda} + \frac{{G_R}^2}{H_R + \lambda} - \frac{{G_I}^2}{H_I + \lambda} \right) - \gamma.$$

Accept splits with sufficient positive gain subject to `min_child_weight`, `max_depth`, etc.

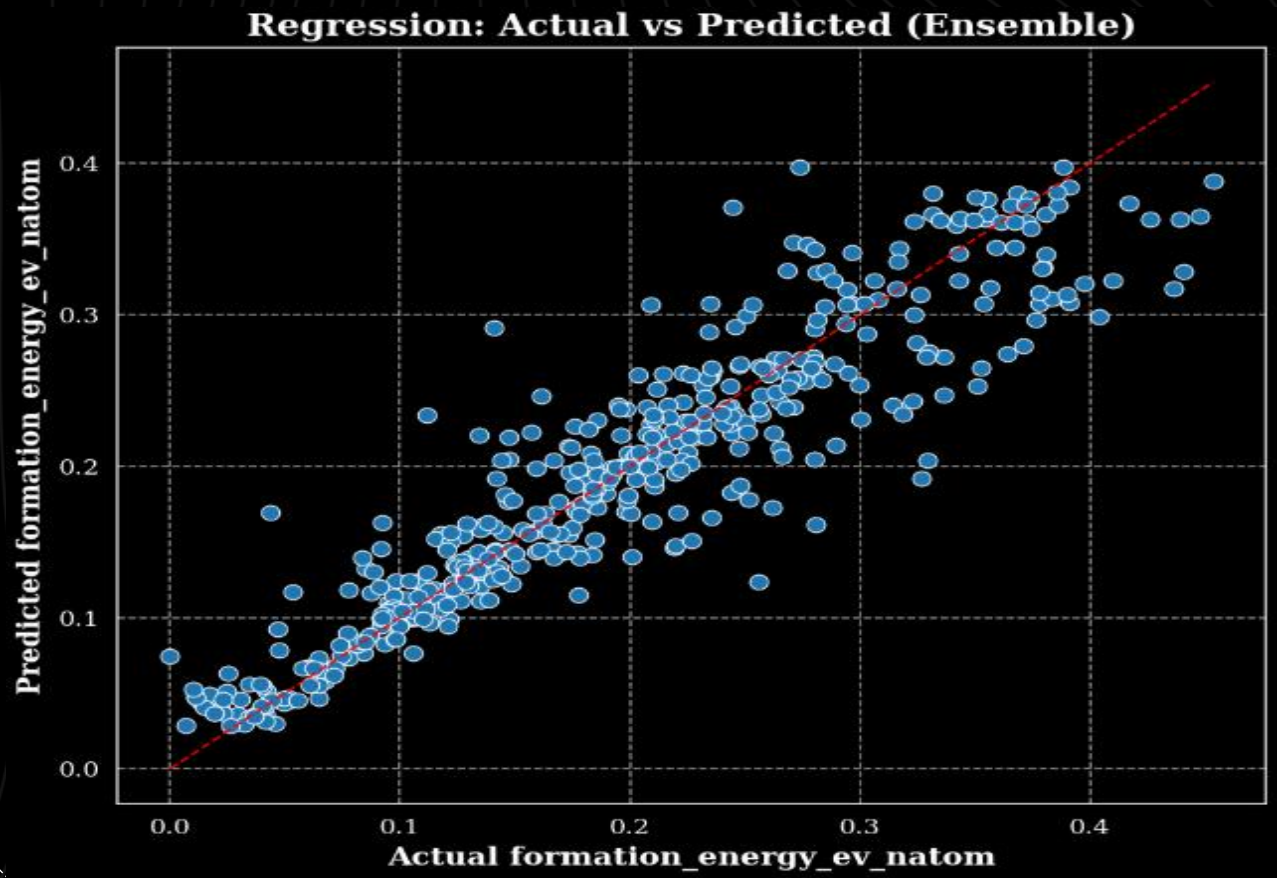
Code Implementation

```
# ------------------------
# Randomized search for XGBoost (regression)
# ------------------------
xgb_reg_sk = XGBRegressor(objective='reg:squarederror', tree_method=tree_method, predictor=predictor, verbosity=0, random_state=42, n_jobs=1)
rand_reg = RandomizedSearchCV(xgb_reg_sk, param_distributions=param_dist_common, n_iter=30, cv=3, scoring='r2', random_state=42, n_jobs=1, verbose=1)
print("Running RandomizedSearchCV for XGB reg (this may take a while)...")
t0 = time.time()
rand_reg.fit(X_train_reg, y_train_reg)
print("XGB reg search done in {:.1f}s. Best params:".format(time.time()-t0), rand_reg.best_params_)
best_reg_params = rand_reg.best_params_.copy()
n_estimators_reg = int(best_reg_params.pop('n_estimators', 300))
# ------------------------
# Train final XGBoost reg with early stopping (xgb.train)
# ------------------------
params_xgb_reg = best_reg_params.copy()
params_xgb_reg.update({'objective':'reg:squarederror','tree_method':tree_method,'verbosity':0,'seed':42})
dtrain = xgb.DMatrix(X_train_reg, label=y_train_reg, feature_names=feature_names)
dval = xgb.DMatrix(X_val_reg, label=y_val_reg, feature_names=feature_names)
dtest = xgb.DMatrix(X_test_reg, label=y_test_reg, feature_names=feature_names)
bst_reg = xgb.train(params_xgb_reg, dtrain, num_boost_round=n_estimators_reg, evals=[(dtrain,'train'),(dval,'valid')], early_stopping_rounds=20, verbose_eval=False)
print("Trained XGB reg. best_iteration:", getattr(bst_reg, "best_iteration", None))

y_test_pred_log_xgb = predict_with_best_ntree(bst_reg, dtest)
```

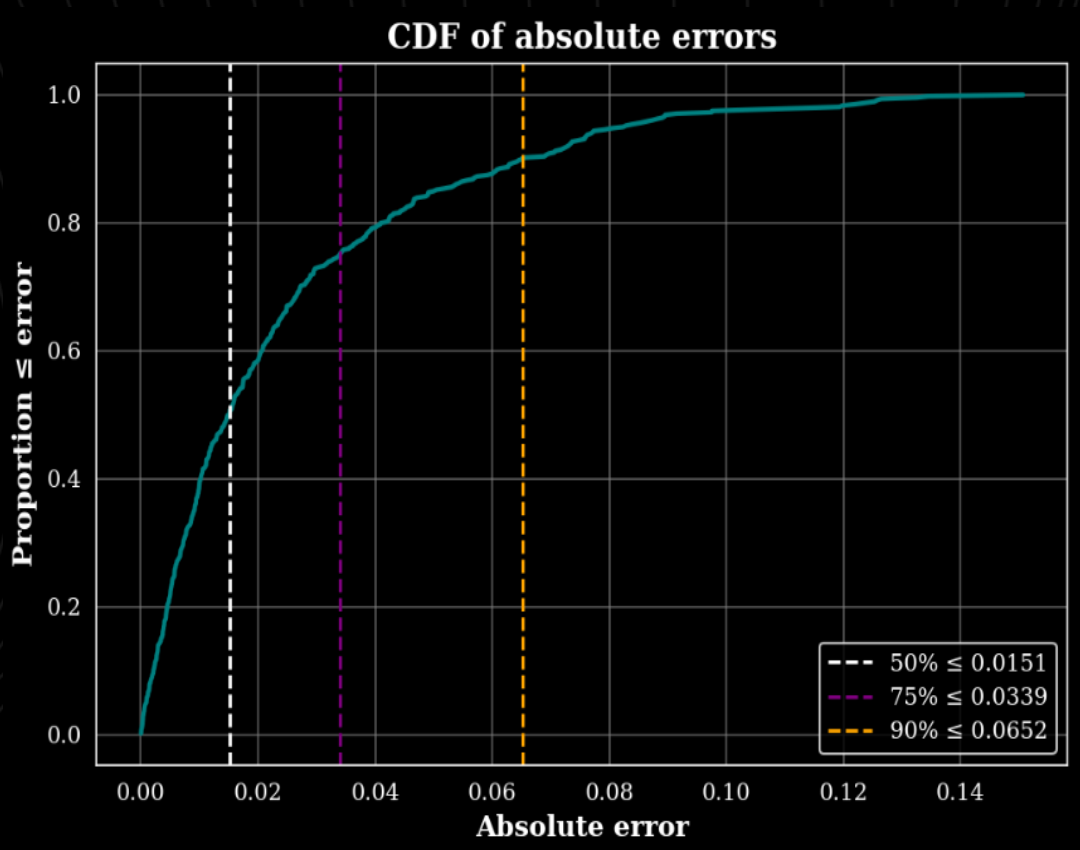
Code Implementation

```
# ------------------------
# Classification Randomized search (XGB)
# ------------------------
xgb_clf_sk = XGBClassifier(objective='binary:logistic', tree_method=tree_method, predictor=predictor, verbosity=0, random_state=42, n_jobs=1, use_label_encoder=False)
rand_clf = RandomizedSearchCV(xgb_clf_sk, param_distributions=param_dist_clf, n_iter=30, cv=StratifiedKFold(n_splits=3), scoring='f1', random_state=42, n_jobs=1, verbose=1)
print("Running RandomizedSearchCV for XGB clf...")
t0 = time.time()
rand_clf.fit(X_train_clf, y_train_clf)
print("XGB clf search done in {:.1f}s. Best params:".format(time.time()-t0), rand_clf.best_params_)
best_clf_params = rand_clf.best_params_.copy()
n_estimators_clf = int(best_clf_params.pop('n_estimators', 300))
# Train final XGB classifier (xgb.train)
params_xgb_clf = best_clf_params.copy()
params_xgb_clf.update({'objective':'binary:logistic','eval_metric':'auc','tree_method':tree_method,'verbosity':0,'seed':42})
dtrain_clf = xgb.DMatrix(X_train_clf, label=y_train_clf, feature_names=feature_names)
dval_clf = xgb.DMatrix(X_val_clf, label=y_val_clf, feature_names=feature_names)
dtest_clf = xgb.DMatrix(X_test_clf, label=y_test_clf, feature_names=feature_names)
bst_clf = xgb.train(params_xgb_clf, dtrain_clf, num_boost_round=n_estimators_clf, evals=[(dtrain_clf,'train'),(dval_clf,'valid')], early_stopping_rounds=20, verbose_eval=False)
print("Trained XGB clf. best_iteration:", getattr(bst_clf, "best_iteration", None))

y_proba_xgb = predict_with_best_ntree(bst_clf, dtest_clf)
```

Results - Regression



RMSE=0.0371, R^2=0.8677



CDF shows the fraction of predictions within different absolute error tolerances.

Results - Classification